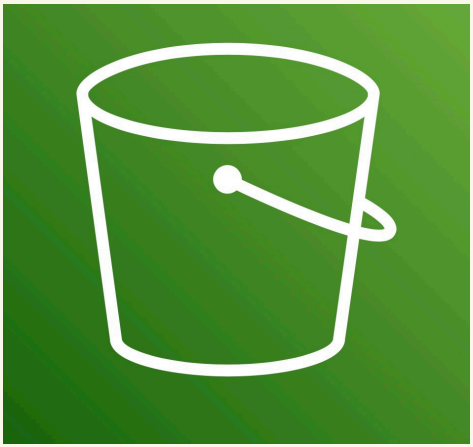
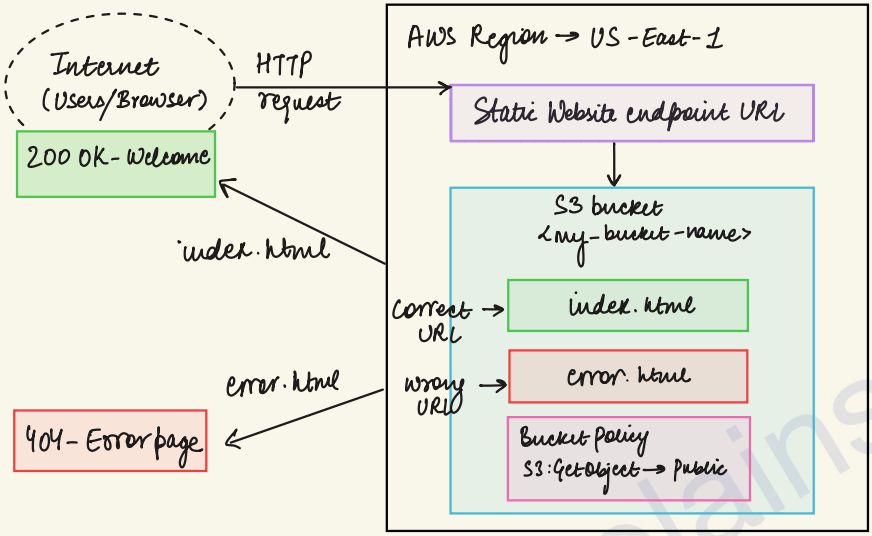
Everything you need to know about -

STATIC WEBSITE HOSTING USING AWS S3



How to Create a Static Website Using S3



Static Websites → They are like a printed book. → different users access the same page → see the same content. The content doesnt change on who you are what time it is or what you click

↓

unless the developer manually edits the file

(HTML, CSS, JS)

HTML → structure, CSS → styling, JS → interactivity

Dynamic Websites → like a restaurant with chef → every user gets a freshly cooked response built on the fly from a database

Static websites are powerful because

1. hosting charges ≈ $0
2. Scaling = automatic → traffic ↑se → AWS handles it on its own ↓ you dont need to do anything
3. fast loading → since so computation happening
4. Server side code ✗ → "hacked" ⓧ through → DB exploits, → Dependency injections

SurbhiExplains

What is Amazon S3 → Simple Storage Service

Its basically a globally distributed hard drive in the cloud
↓
not like your computer's Hard drive
↓
its an object storage system

Amazon S3

Bucket
<bucket name>
globally unique name

- index.html
  Key (name) + content
- error.html
  Key (name) + content
- logo.png
  Key (name) + content

each file = 1 object

Bucket (Backup)
<Backup bucket name>
↓
different bucket, same account

- db-backup-2024.zip
  (upto 5TB per object)

Key facts

11-9's durability
99.999999999 %

- max object size: 5TB
- Region specific but globally named
- pay per GB stored
  +
  per request

SubhiExplains

Bucket → Top level container → Name must be globally unique across all AWS customers worldwide

Object → any file you store inside a bucket.
→ every object has a → Key (its name/path)
→ actual content

Region → physical AWS data center location (eg. us-east-1 = N. Virginia, USA)

ARN → Amazon resource name → every AWS resource gets a unique Id.

Public access, ACL's & Bucket policies → The Security layer

S3 access control : 3 layers

Layer 1 → Block public access (master switch)

when ON (default) → No public access regardless of other settings

This lab → Turn OFF to allow public access

Layer 2 → ACL = Access control lists (legacy)

Old per-object permission system. This lab : ACL (disabled)

Modern AWS replaces ACL's with Bucket policies (simple, more powerful)

Layer 3 → Bucket policy (JSON rules)

Defines exactly WHO can do WHAT to WHICH objects

This lab : allows everyone (*) to getObject (read files) from bucket

↔ All the 3 layers must allow access for a public request to succeed.

Bucket Policy → JSON

```
{
    "Id": "Policy1",
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "Statement1",
            "Action": ["s3:GetObject"],
            "Effect": "Allow",
            "Resource": "arn:aws:s3:::surbhibucket1230/*",
            "Principal": "*"
        }
    ]
}
```

IAM policy language version → always use this

just a label for the rule

allow reading or downloading objects

wildcard (/*) → apply to all objects in bucket

permit this action → could also be Deny

everyone → any user, anywhere, no login required

SurbhiExplains

How static website hosting works in S3

↓

when you enable static website hosting → S3 stops being a file storage system and starts behaving like a web server

Browser (user visits URL) —HTTP GET→ S3 Website Endpoint

http://<bucketname>.s3-website.<Region>.amazonaws.com

↓

does path exist? (check bucket objects)

Yes → Serve index.html
200 OK response
↓
HTML sent to browser
"Hey welcome"

No → Serve error.html
404 Not found response
↓
Error HTML sent to browser
"Page not found"

(root URL) Special rule → visiting / automatically serves index.html

important ⇒ The endpoint URL you get after enabling static website hosting is Http and not Https

↓

S3 static website hosting doesn't natively support SSL/TLS certs.

↓

for https:// you need to put AWS Cloudfront (CDN) in front

SurbhiExplains

Steps of labs

Step 1 → Sign in to AWS and set the region you want

Step 2 → Create S3 bucket
- ↳ unique name, ACL's off
- ↳ uncheck block all public access

Step 3 → Set bucket policy → Permissions → Bucket policy (JSON)
- ↓ Should be updated with your bucket's ARN

Allow public → S3: GetObject

Step 4 → Enable Static Hosting
- ↳ Properties → Static website hosting → edit → enable
- ↓ Set index.html and error.html

Also now upload your HTML files
- index.html
- error.html

Step 5 → Test
- → Correct URL ✓ → content of index.html should be visible
- → Wrong URL ✗ → content of error.html should be visible

What we have built

↳ globally accessible static website running on AWS infrastructure
- ↳ cost = ~$0
- ↳ uptime = 99.99%+
- ↳ scalability = unlimited

↓ No servers to manage, no OS to patch, no database to secure

SurbhiExplains

Some Beyond the lab content

① HTTPS with Cloudfront → S3 static hosting does only HTTP
↓
In production → you'd put AWS Cloudfront (CDN) infront of S3.
↓
Cloudfront gives you free SSL/TLS certificates via AWS Certificate Manager
↓
it also caches your content at 450+ edge locations worldwide (latency ↓↓↓)

② Custom Domain Names → Instead of the long .amazonaws.com URL
↓
Use Route 53 (AWS's DNS service) to point
www.yourwebsite.com → Cloudfront → S3

③ CI/CD deployments → Use Github actions or AWS CodePipeline to automatically sync your code to S3 everytime you push a git commit.
↓
Manual uploads ⓧ

④ S3 Versioning → you can enable versioning on a bucket so every version of every file is preserved.
↓
If you accidently overwrite index.html
↓
you can restore the previous version instantly

⑤ Lifecycle policies → Rules that automatically delete/archive old objects after X days
↓
crucial for managing costs on buckets that accumulate logs or backups

SurbhiExplains